University of Exeter

ECM2433: The C Family

Pointers and function parameters

Recap: Pointers

University of Exeter

- Pointer variables can contain the memory address of other variables
- `<datatype>  *p;`

- Allows dynamic memory allocation.
  - Care needed to avoid memory leaks and segmentation violations
- Can be very fast.
- Powerful, but can lead to obscure programs.

Functions: pass by value

```
#include <stdio.h>

void func(char);

int main ()
{
   char c = 'A';
   printf("Before:  c = %c   &c = %p\n", c, &c);
   func(c);
   printf("After:   c = %c   &c = %p\n",c, &c);
}

void func(char ch)
{
    ch = 'B';
    printf("func:    ch = %c &ch = %p\n", ch, &ch);
}
```

```
Before:  c = A   &c = 0x30cb80cdf
func:    ch = B &ch = 0x30cb80cbf
After:   c = A   &c = 0x30cb80cdf
```

The value of c is not updated because `ch` is a copy of `c`.

Functions: pass by reference

```
#include <stdio.h>

void func(char *);

int main ()
{
   char c = 'A';
   printf("Before:  c = %c   &c = %p\n", c, &c);
   func(&c);
   printf("After:   c = %c   &c = %p\n",c, &c);
}

void func(char *ch)
{
    *ch = 'B';
    printf("func:   *ch = %c  ch = %p  &ch = %p\n", *ch, ch, &ch);
}
```

```
Before:  c = A   &c = 0x30dac0cef
func:   *ch = B  ch = 0x30dac0cef  &ch = 0x30dac0cc8
After:   c = B   &c = 0x30dac0cef
```

In order to update the value of a parameter within a function it must be passed as a pointer.

The value of variables passed in cannot be changed.

Here the <u>memory address</u> where c is stored is copied, (and is not updated), not c's value

Functions and arrays

```
#include <stdio.h>
#include <stdlib.h>

int *doubler(int *x, int len);
void Xdoubler(int *x, int len);
void printArray(int *x, int len);

int main()
{
    int i, n = 10, *array, *q;

    if (!(array = (int *)malloc(n*sizeof(int)))) {
        printf("malloc failed\n");
        exit(1);
    }

    for (i = 0; i < n; i++)
        array[i] = i;

    printf("array = %p    ", array);
    printArray(array, n);
```

```
array = 0x60000340c270      0  1  2  3  4  5  6  7  8  9
```

```
void printArray(int *a, int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("%3d", a[i]);
    printf("\n");
}
```

Functions and arrays

```
    q = doubler(array, n);
    printf("array = %p    ", array);
    printArray(array, n);
    printf("q = %p    ", q);
    printArray(q, n);

    Xdoubler(array, n);
    printf("array = %p    ", array);
    printArray(array, n);
}

int *doubler(int *x, int len)
{
    int i;

    printf("doubler: x = %p\n", x);
    for (i = 0; i < len; i++)
        x[i] *= 2;
    return x;
}
```

```
doubler: x = 0x60000340c270
array = 0x60000340c270      0  2  4  6  8 10 12 14 16 18
q = 0x60000340c270      0  2  4  6  8 10 12 14 16 18
```

Functions and arrays

```
    q = doubler(array, n);
    printf("array = %p     ", array);
    printArray(array, n);
    printf("q = %p     ", q);
    printArray(q, n);

    Xdoubler(array, n);
    printf("array = %p     ", array);
    printArray(array, n);
}
```

```
doubler: x = 0x60000340c270
array = 0x60000340c270      0  2  4  6  8 10 12 14 16 18
q = 0x60000340c270      0  2  4  6  8 10 12 14 16 18
```

```
Xdoubler: x = 0x6000038dc270
array = 0x6000038dc270      0  4  8 12 16 20 24 28 32 3
```

```
int *doubler(int *x, int len)
{
    int i;

    printf("doubler: x = %p\n", x);
    for (i = 0; i < len; i++)
        x[i] *= 2;
    return x;
}
```

```
void Xdoubler(int *x, int len)
{
    int i;

    printf("Xdoubler: x = %p\n",
    for (i = 0; i < len; i++)
        *x++ *= 2;
    return;
}
```

Functions: pass by reference

```
#include <stdio.h>
#include <string.h>

void func (char *s);

int main ()
{
   char *p;
   func(p);
   printf("p = %s\n", p);
}

void func(char *str)
{
   /* strcpy(char *dest, char *src) */
   strcpy(str, "Hello");
}
```

What happens?

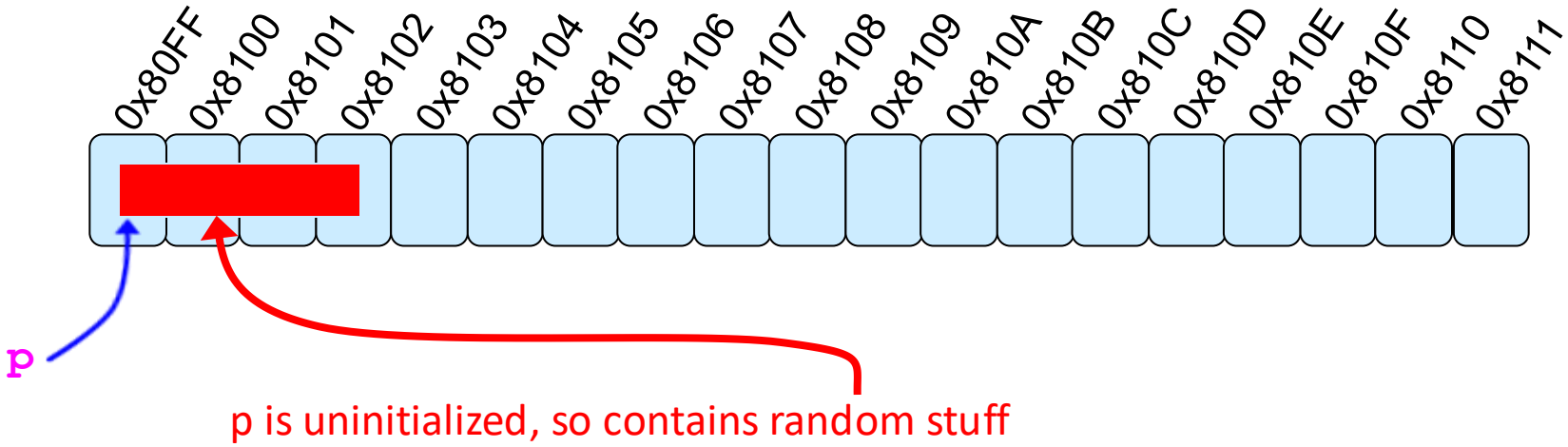
Functions: pass by reference

```
#include <stdio.h>
#include <string.h>

void func (char *s);

int main ()
{
   char *p;
   func(p);
   printf("p = %s\n", p);
}

void func(char *str)
{
   /* strcpy(char *dest, char *src) */
   strcpy(str, "Hello");
}
```

```
$ pass-by-ref
 [1]    77244 bus error  pass-by-ref
```

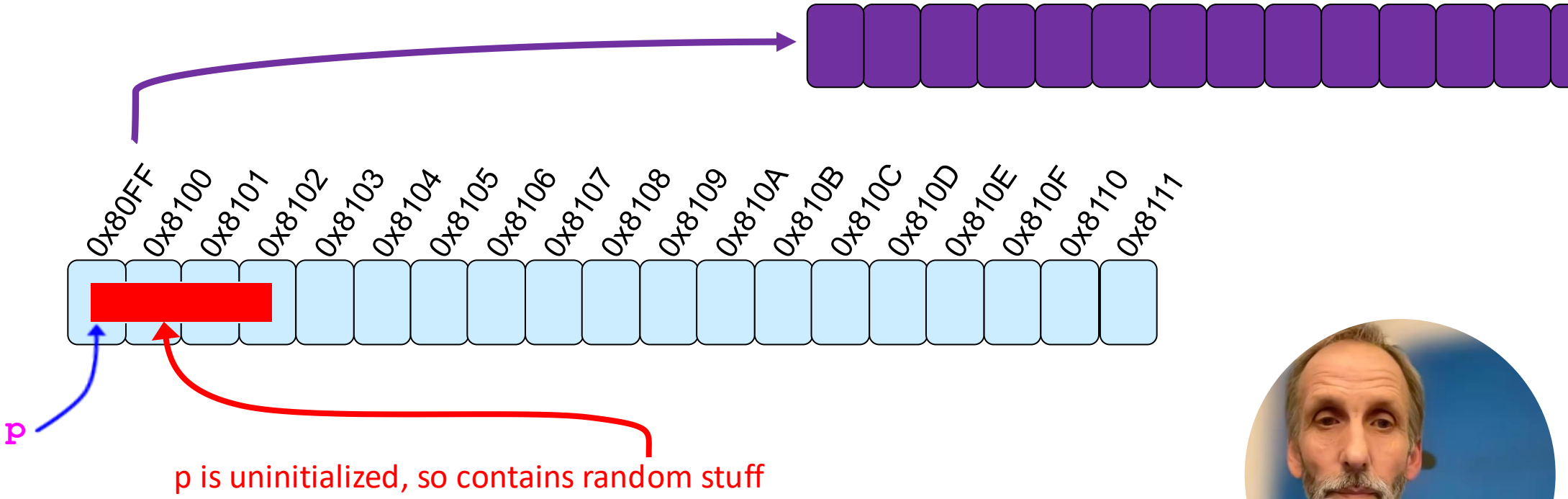
Functions: pass by reference

```
char  *p;
```



Functions: pass by reference

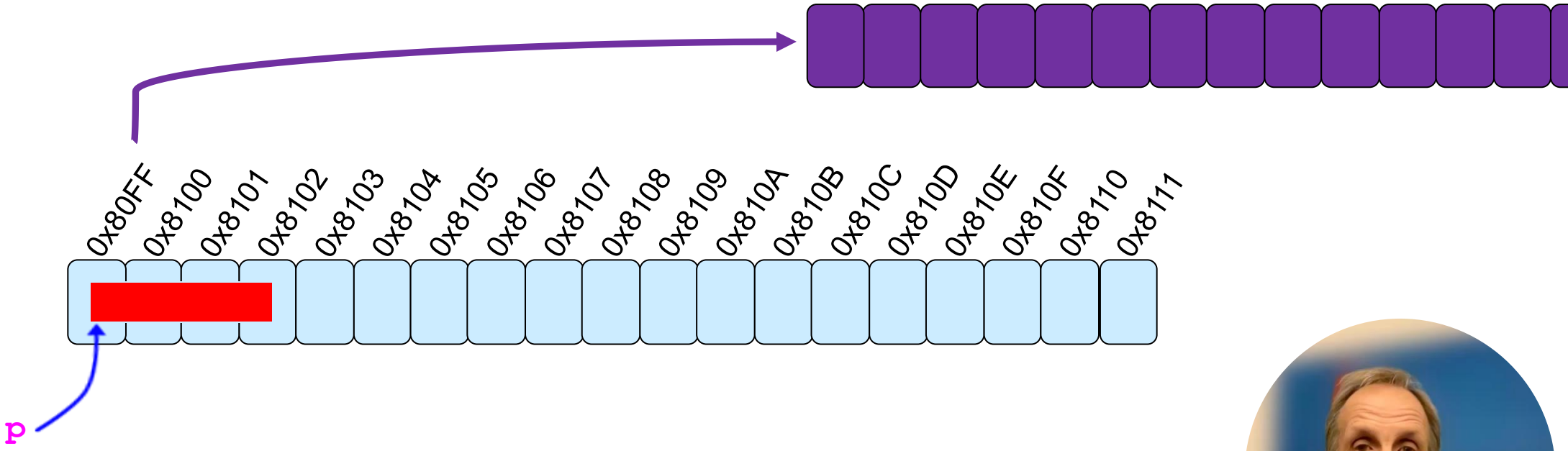
```
char  *p;
```



Functions: pass by reference

```
char  *p;
func(p);
```

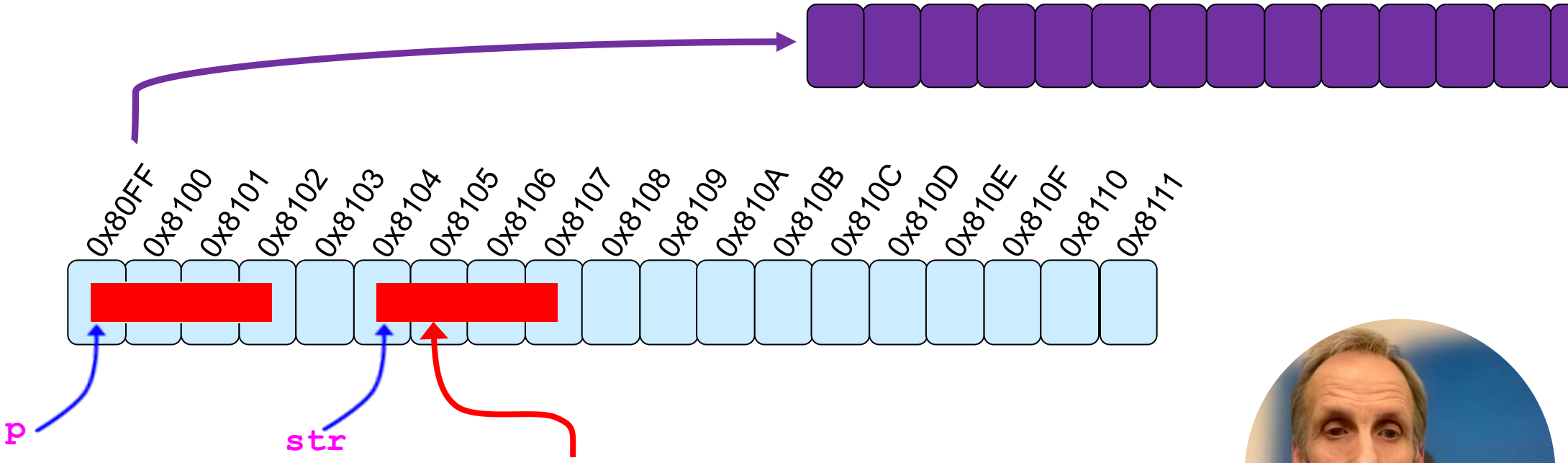
```
void func(char *str)
```



Functions: pass by reference

```
char  *p;
func(p);
```

```
void func(char *str)
```

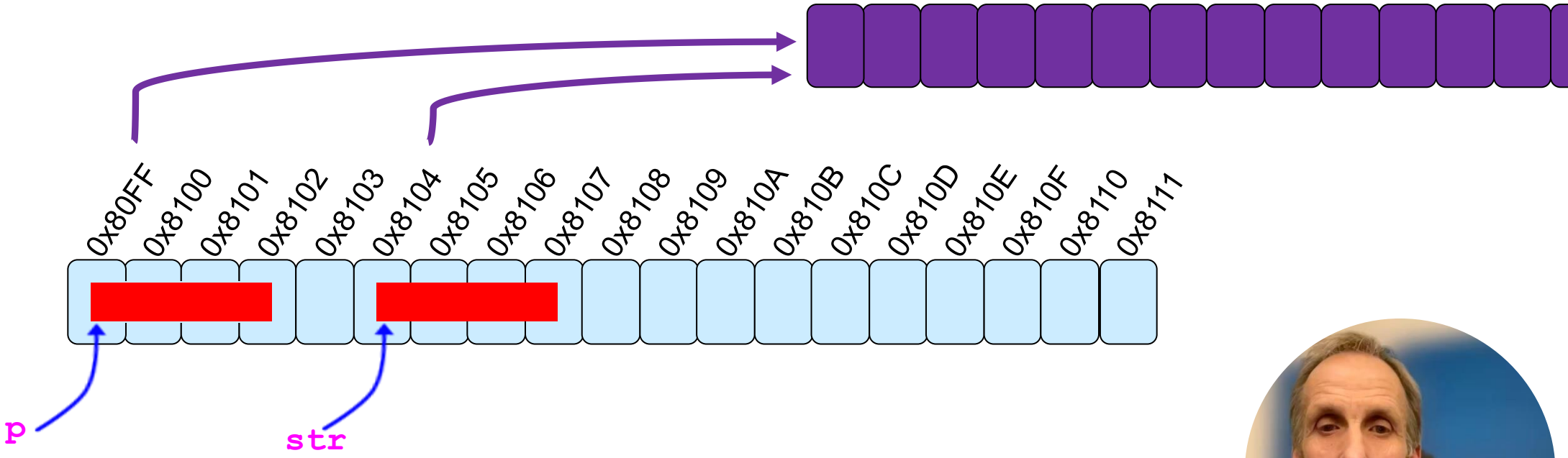


the random stuff is copied from p into str
when the function is called

Functions: pass by reference

```
char  *p;
func(p);
```

```
void func(char *str)
```

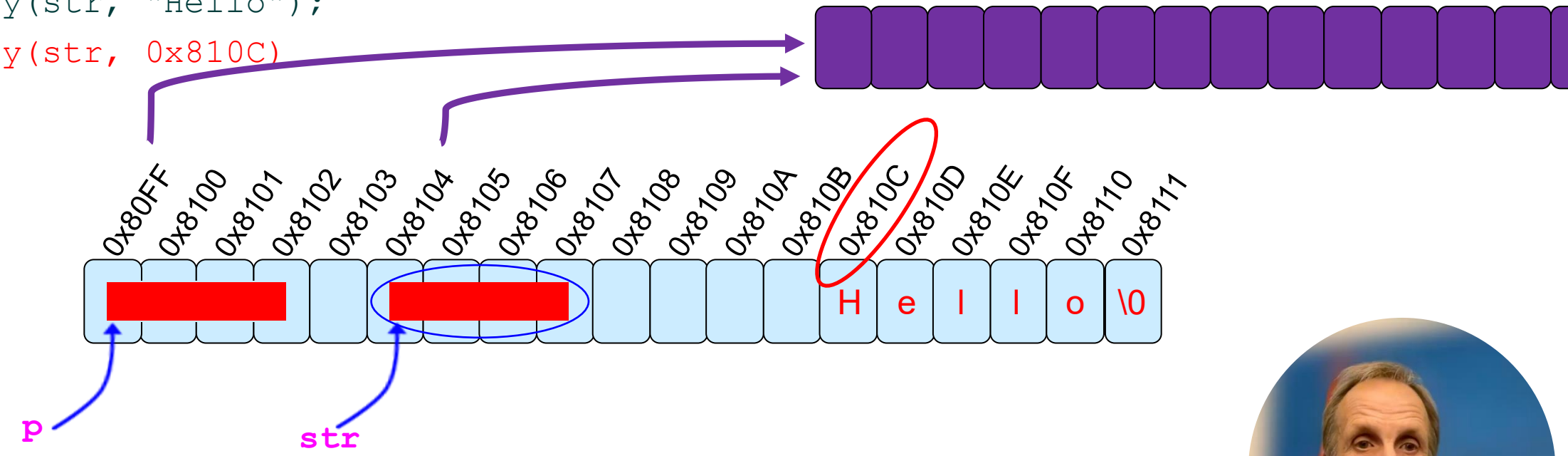


Functions: pass by reference

```
char  *p;
func(p);
```

```
void func(char *str)
strcpy(str, "Hello");
```

```
strcpy(str, 0x810C)
```



Functions: pass by reference

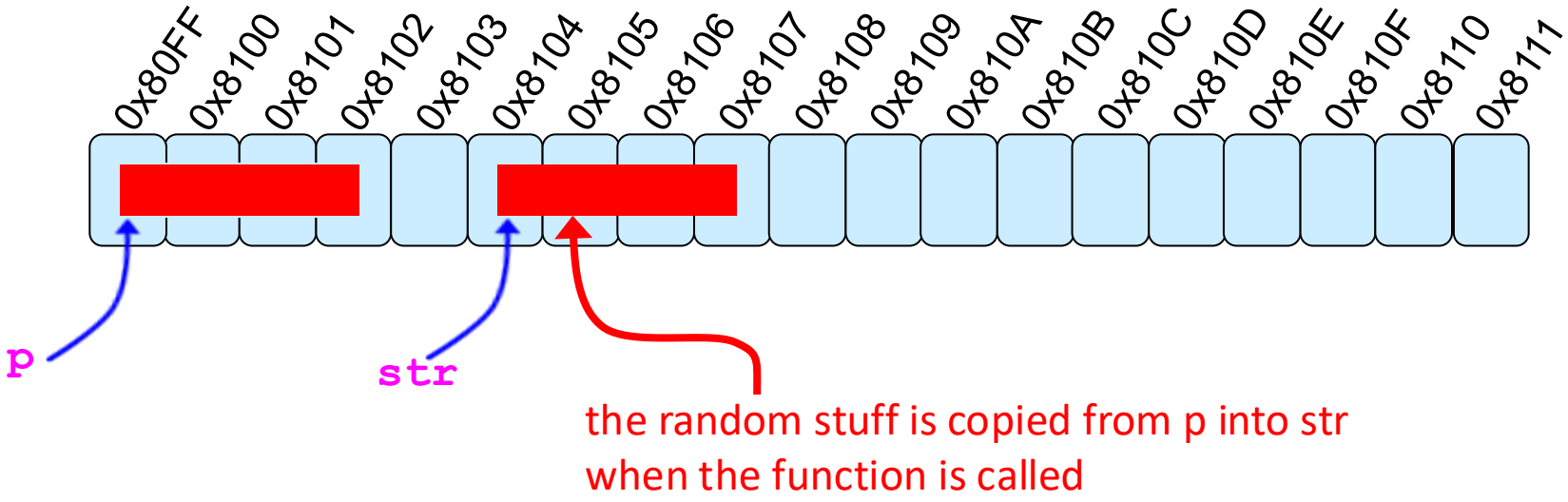
```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void func (char *);
int main ()
{
    char *p;
    func(p);
    printf("p = %p: %s\n", p, p);
}

void func(char *str)
{
    if ( !(str = (char *)malloc(6)) )
    {
        printf("Insufficient memory\n");
        exit(-1);
    }
    strcpy(str, "Hello");
    printf("str = %p: %s\n", str, str);
}
```

```
str = 0x6000032d8040: Hello
p = 0x102ee8ec0: UH□□H□□H□}□□
```

`malloc` changes the value of `str`, but that does not change the value of `p`.

Functions: pass by reference

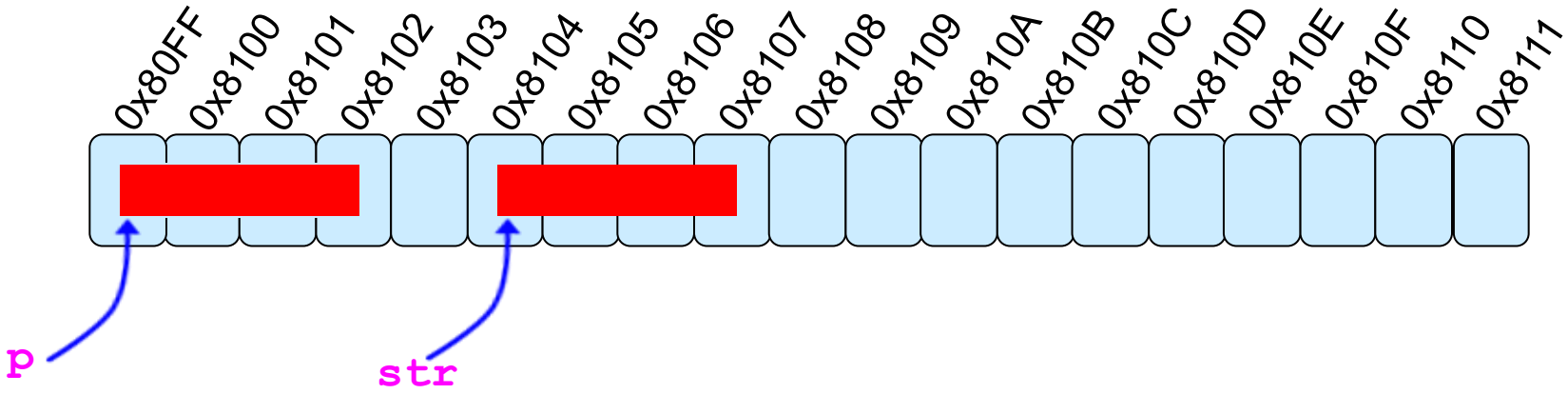
```
char  *p;
func(p);

void func(char *str)
```



Functions: pass by reference

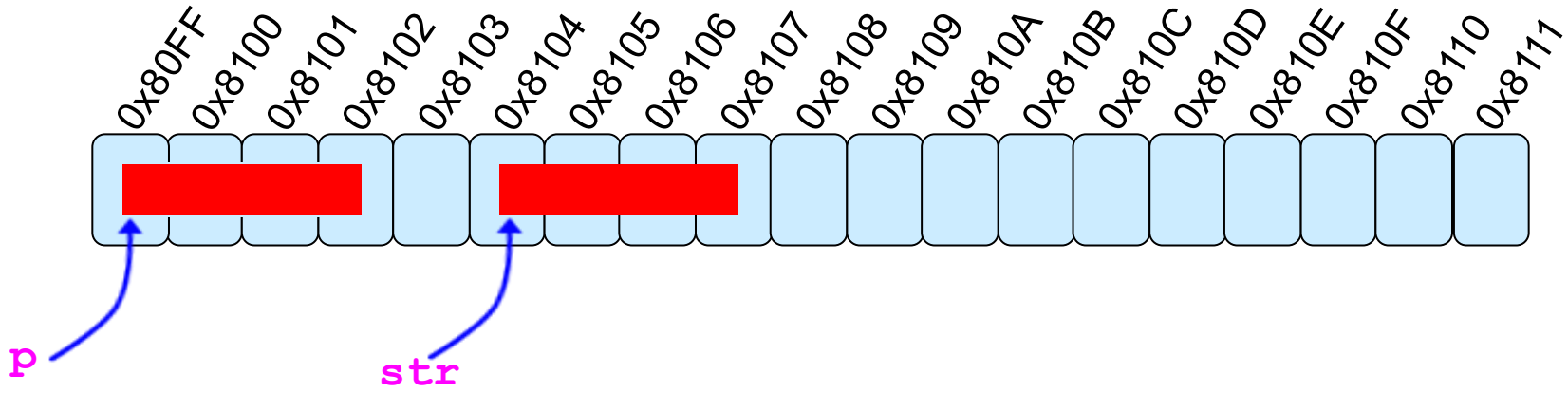
```
char  *p;
func(p);
```

```
void func(char *str)
```



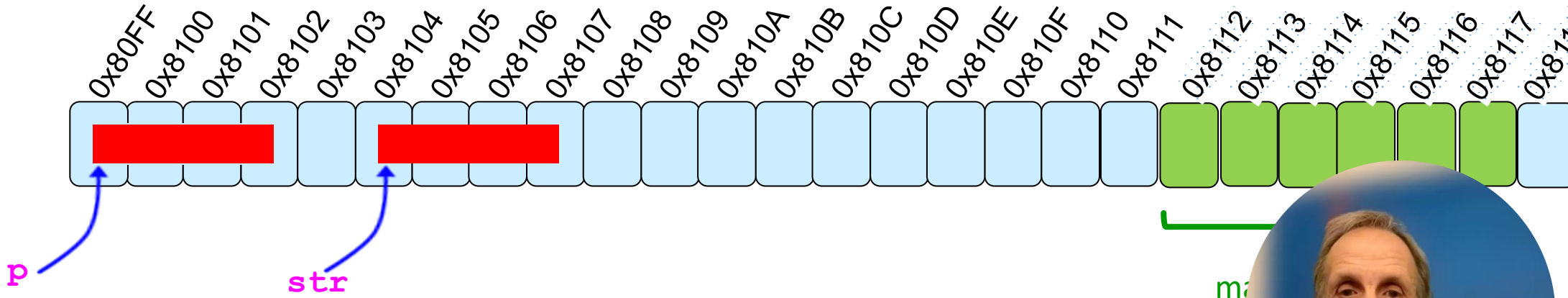
Functions: pass by reference

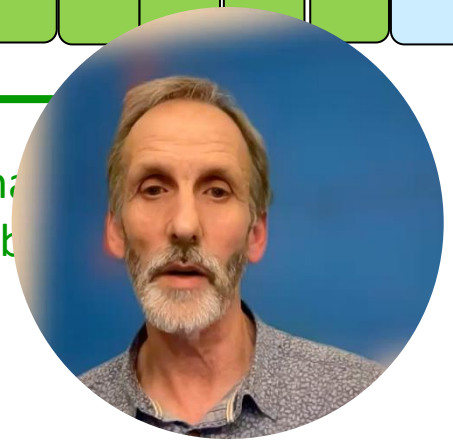
```
char  *p;
func(p);

void func(char *str)
str = (char *)malloc(6);
```



Functions: pass by reference

```
char  *p;
func(p);

void func(char *str)
  str = (char *)malloc(6);
```



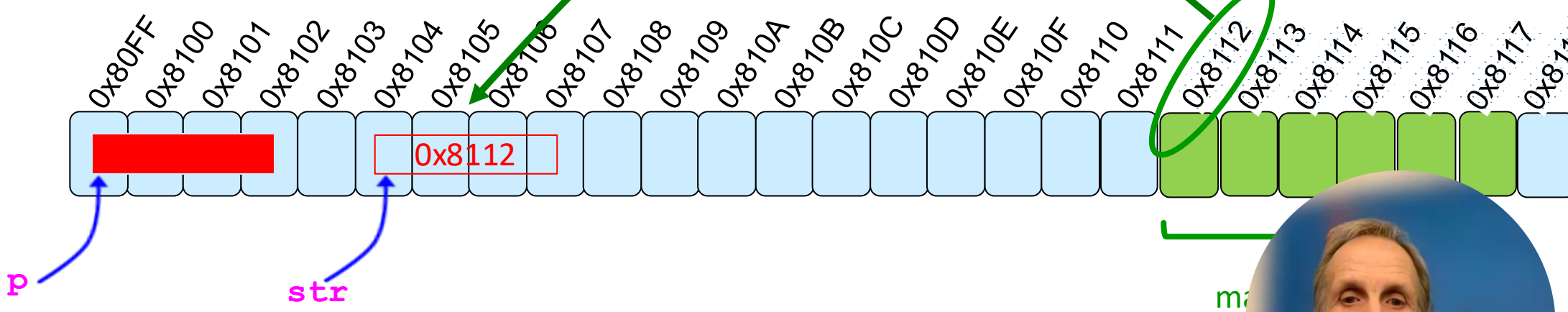


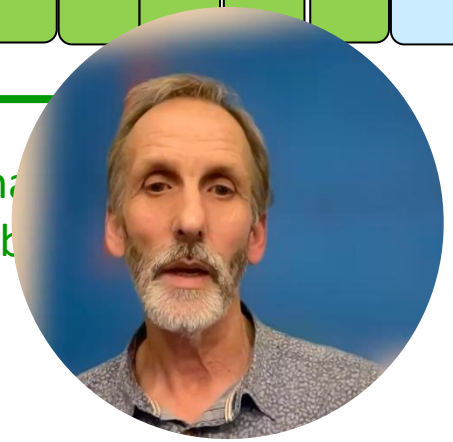
Functions: pass by reference

```
char  *p;
func(p);
```

```
void func(char *str)
  str = (char *)malloc(6);
```

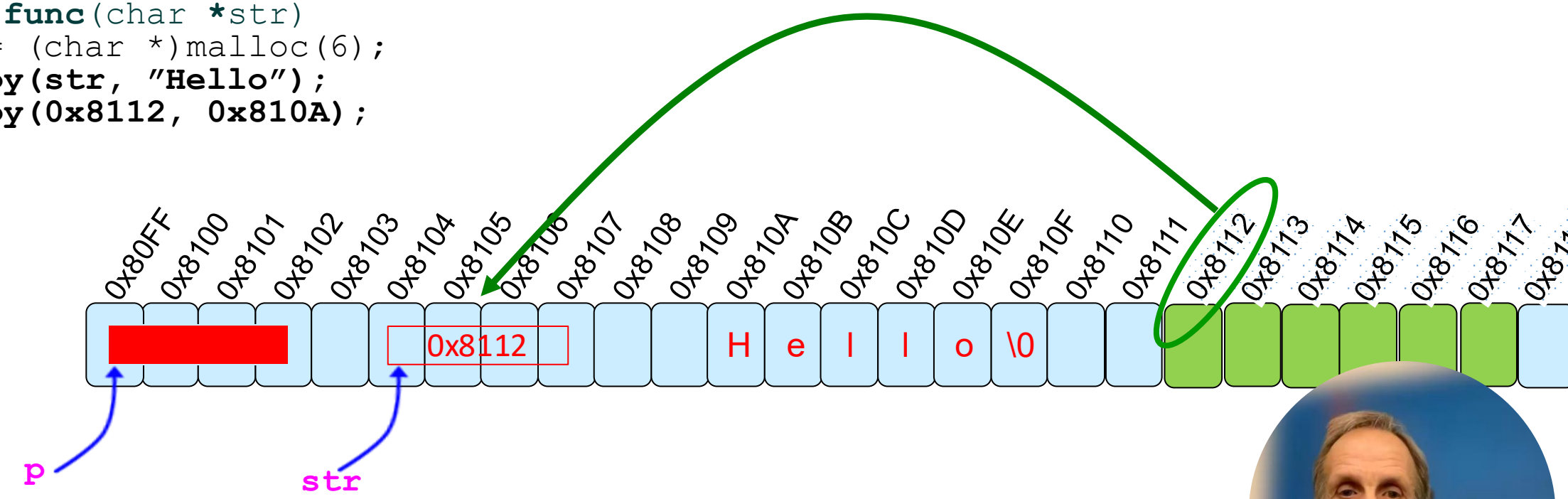
the memory address of the first byte is stored in str

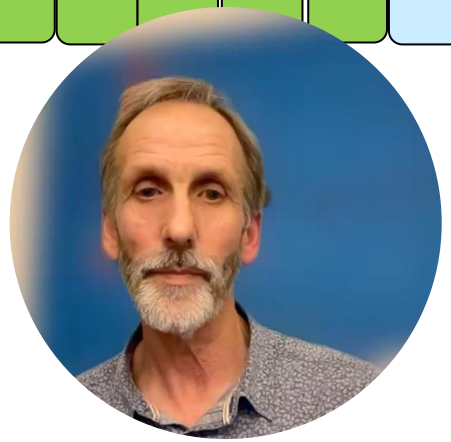




Functions: pass by reference

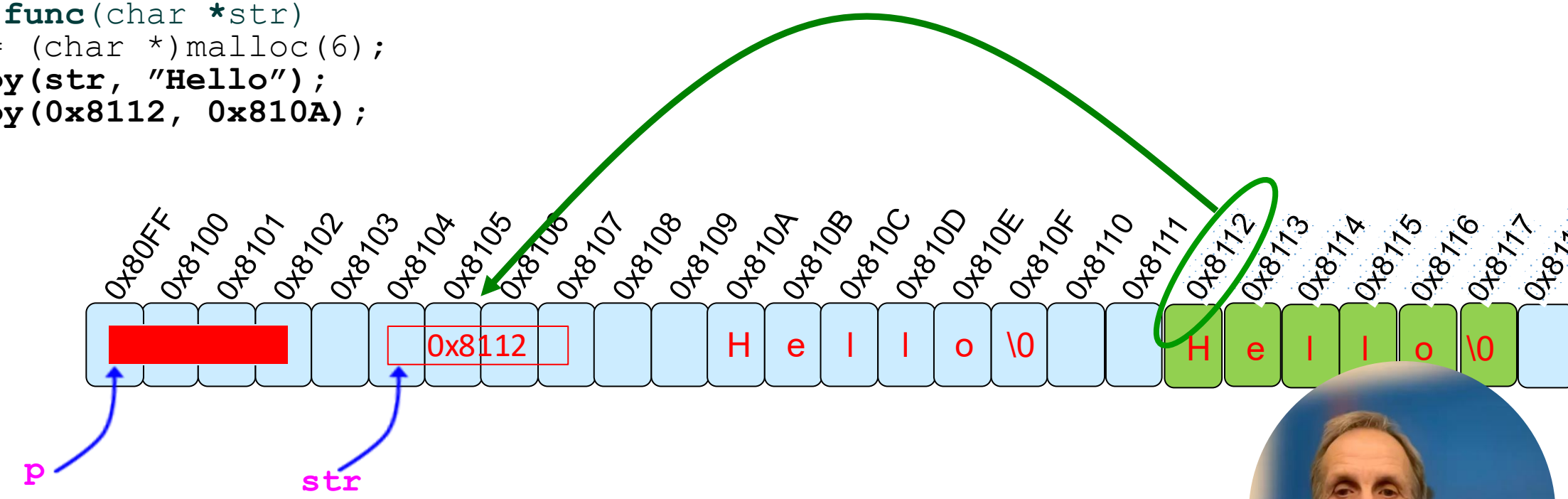
```
char  *p;
func(p);

void func(char *str)
  str = (char *)malloc(6);
  strcpy(str, "Hello");
  strcpy(0x8112, 0x810A);
```





Functions: pass by reference

```
char  *p;
func(p);

void func(char *str)
  str = (char *)malloc(6);
  strcpy(str, "Hello");
  strcpy(0x8112, 0x810A);
```



So **str** is correctly updated,
but **p** is not changed

Pointers to pointers

A pointer is just another a datatype, so you can have a pointer to a pointer
(and a pointer to a pointer to a pointer ...):

```
char   ch = 'A';
char  *p  = &ch;
char **q  = &p;
printf("The value of ch is: %c, %c, %c\n", ch, *p, **q);
```

```
The value of ch is: A, A, A
```

```
printf("The memory address of ch is: %p, %p, %p\n", &ch, p, *q)
```

```
The memory address of ch is: 0x7ffe44c2ed4f, 0x7ffe44c2ed4f, 0x7ffe44
```

Functions: pass by reference

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void setString(char **);

int main()
{
    char *p;
    setString(&p);
    printf("p = %p: %s\n", p, p);
}

void setString(char **str)
{
   if ( !(*str = (char *)malloc(6)) ) {
      printf("Insufficient memory\n");
      exit(-1);
   }
   strcpy(*str, "Hello");
   printf("*str = %p: %s\n", *str, *str);
}
```

Attempt 3:

Pass the address of p

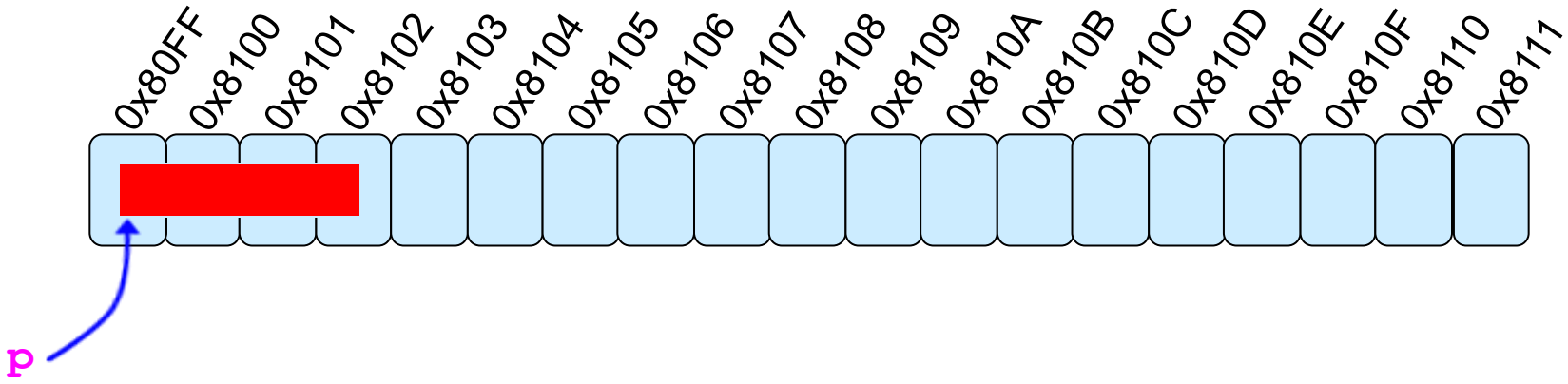
Functions: pass by reference

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void setString(char **);

int main()
{
    char *p;
    setString(&p);
    printf("p = %p: %s\n", p, p);
}

void setString(char **str)
{
   if ( !(*str = (char *)malloc(6)) ) {
      printf("Insufficient memory\n");
      exit(-1);
   }
   strcpy(*str, "Hello");
   printf("*str = %p: %s\n", *str, *str);
}
```

```
*str = 0x600001e74040: Hello
p = 0x600001e74040: Hello
```

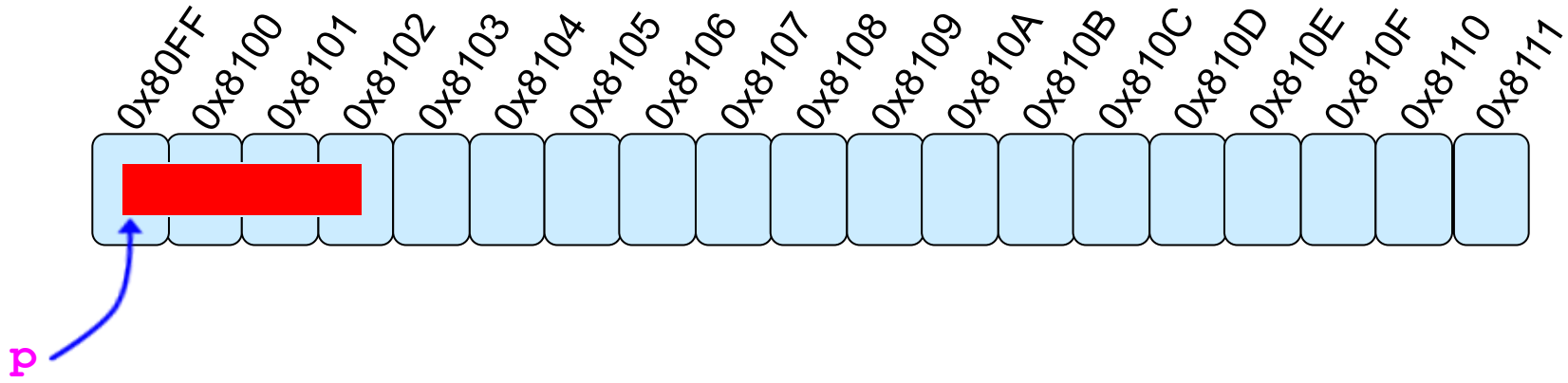
Functions: pass by reference

```
char  *p;
```



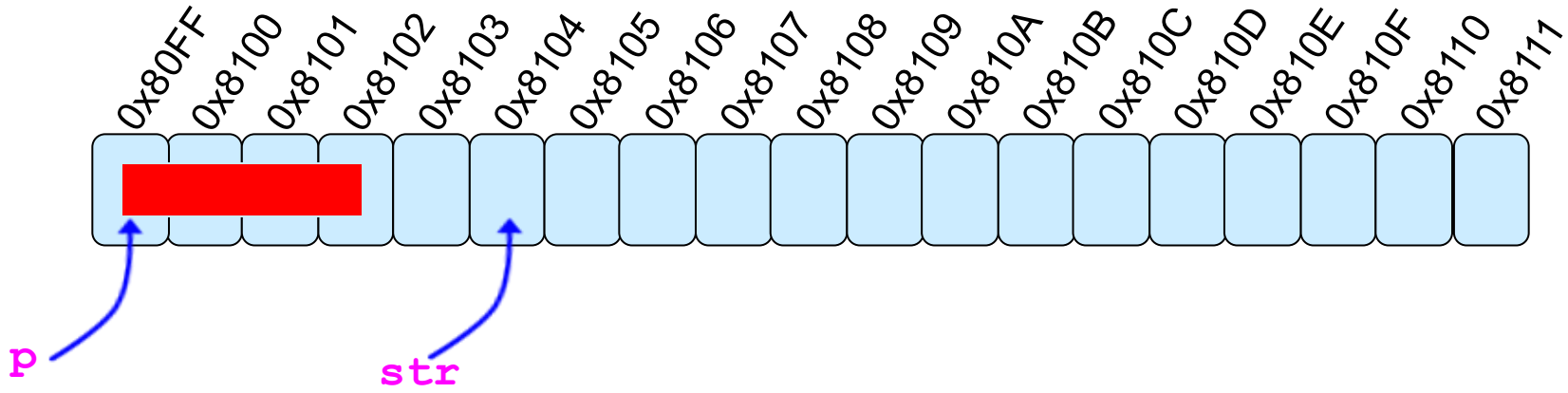
Functions: pass by reference

```
char  *p;
setString(&p);
```



Functions: pass by reference

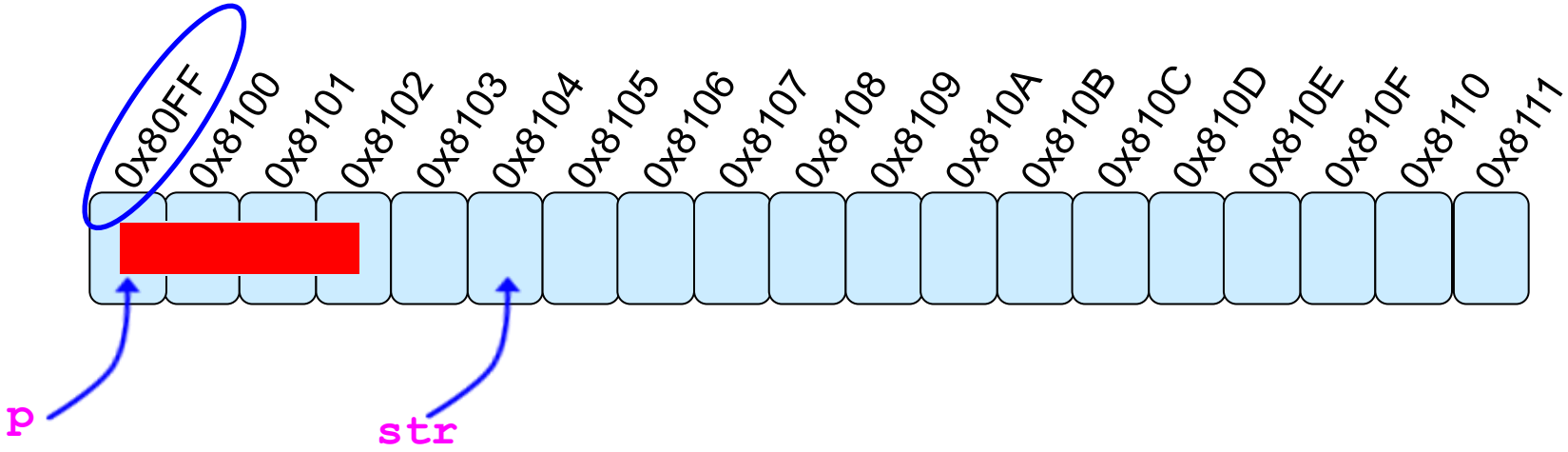
```
char  *p;
setString(&p);

void setString (char **str)
```



Functions: pass by reference

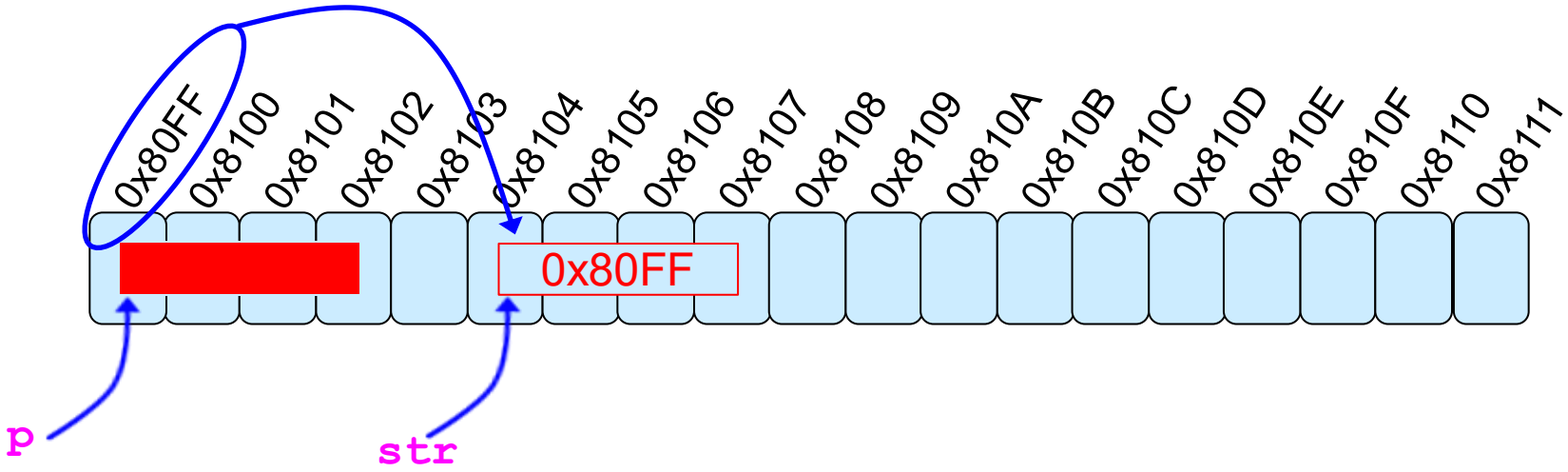
```
char  *p;
setString(&p);
```

```
void setString (char **str)
```



Functions: pass by reference

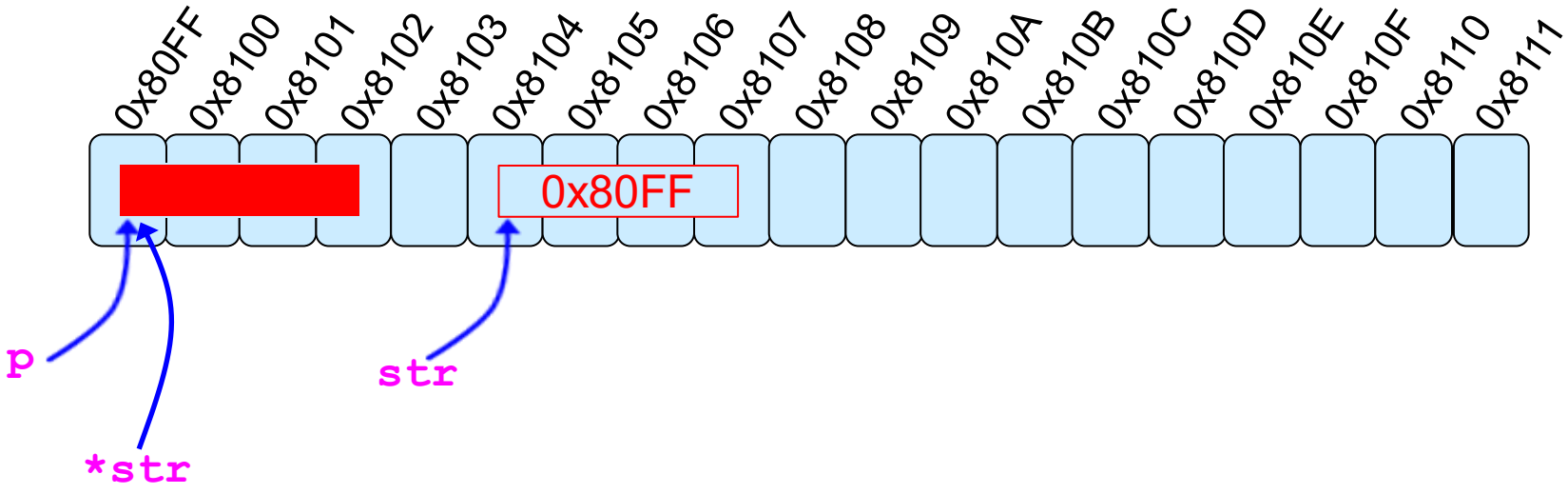
```
char  *p;
setString(&p);

void setString (char **str)
```



Functions: pass by reference

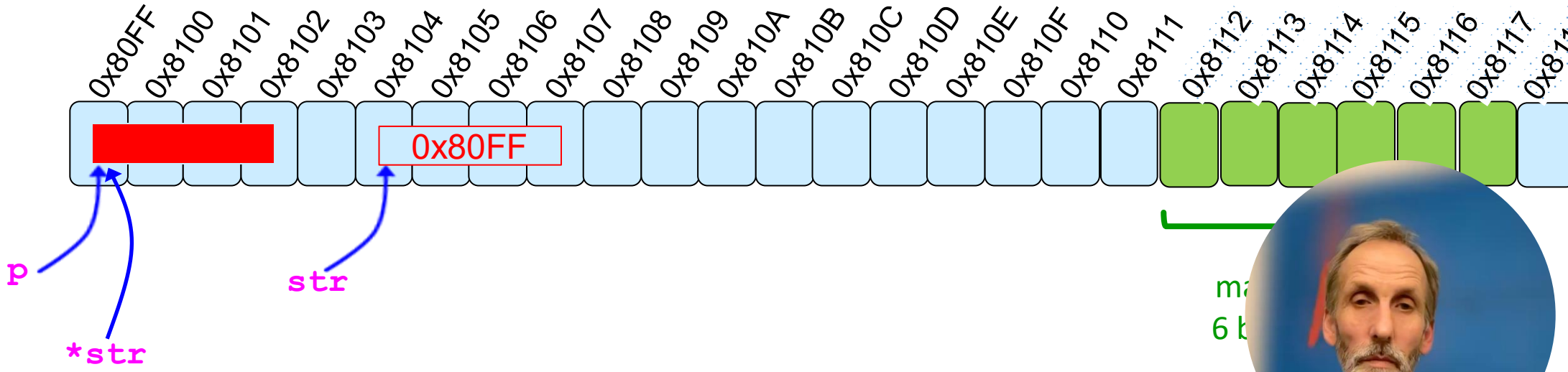
```
char  *p;
setString(&p);
```

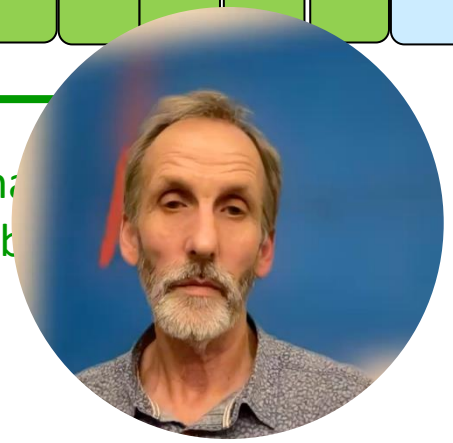
```
void setString(char **str)
```



Functions: pass by reference

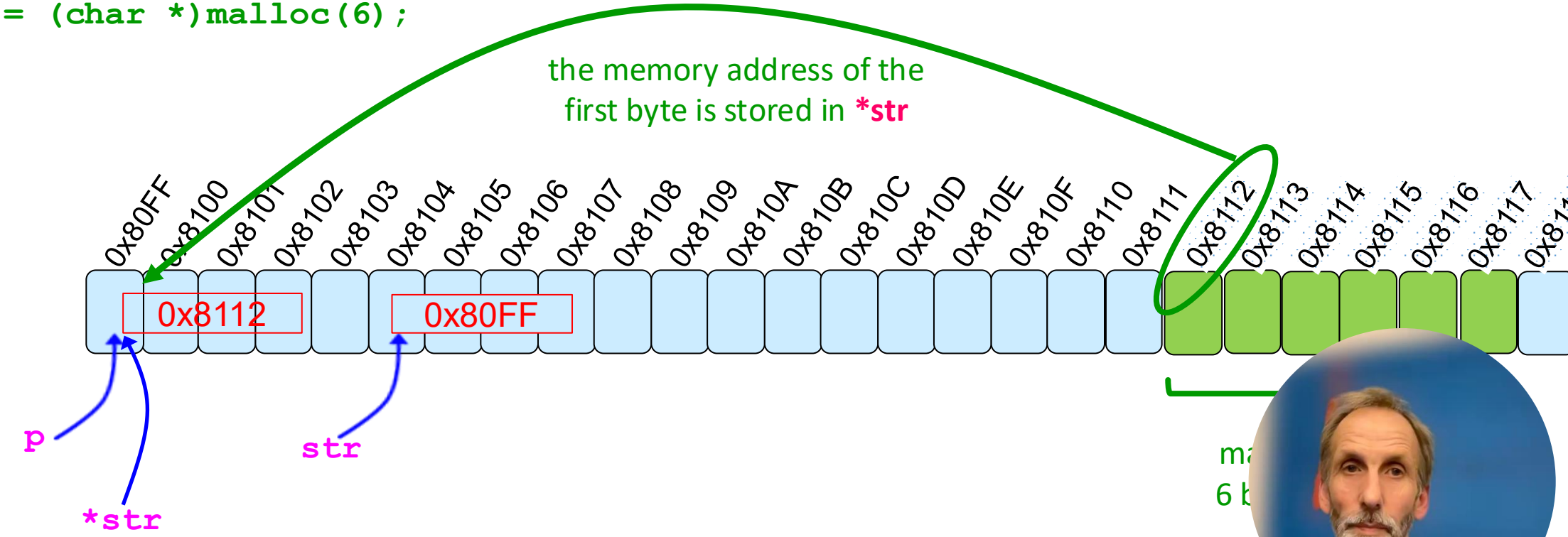
```
char  *p;
func(p);

void setstring(char *str)
*str = (char *)malloc(6);
```

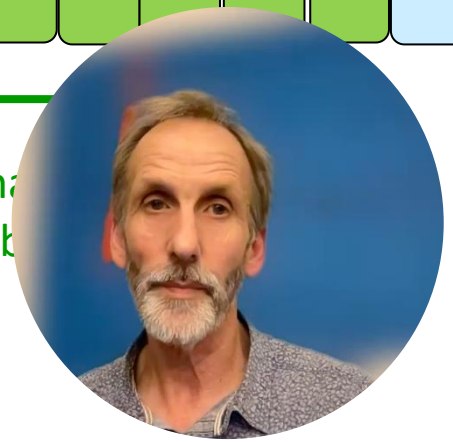




Functions: pass by reference

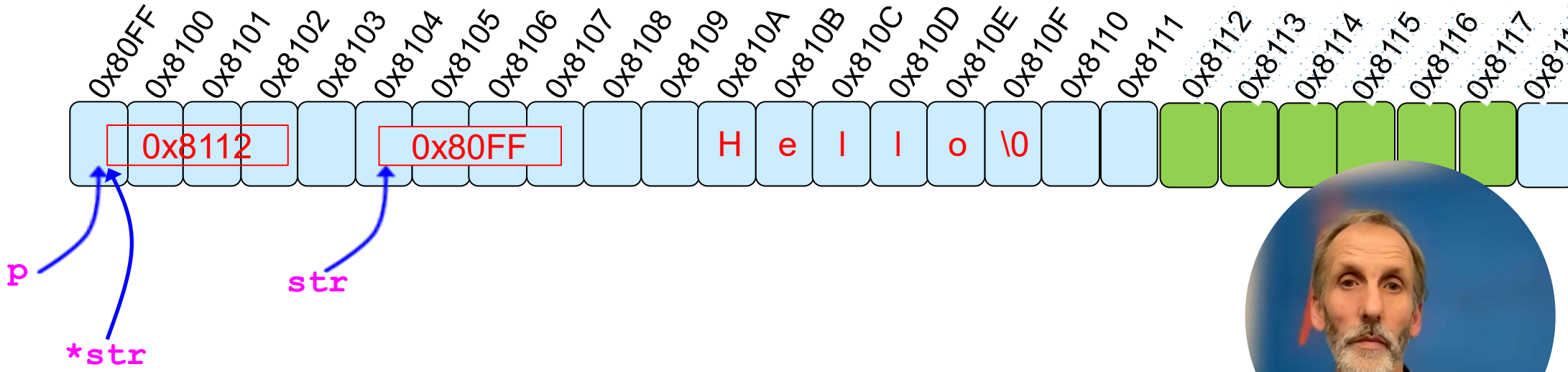
```
char  *p;
func(p);

void setstring(char *str)
*str = (char *)malloc(6);
```

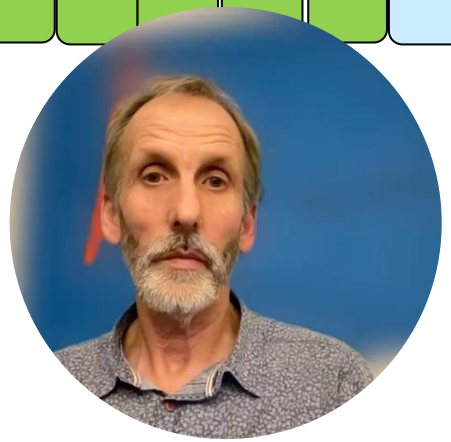




Functions: pass by reference

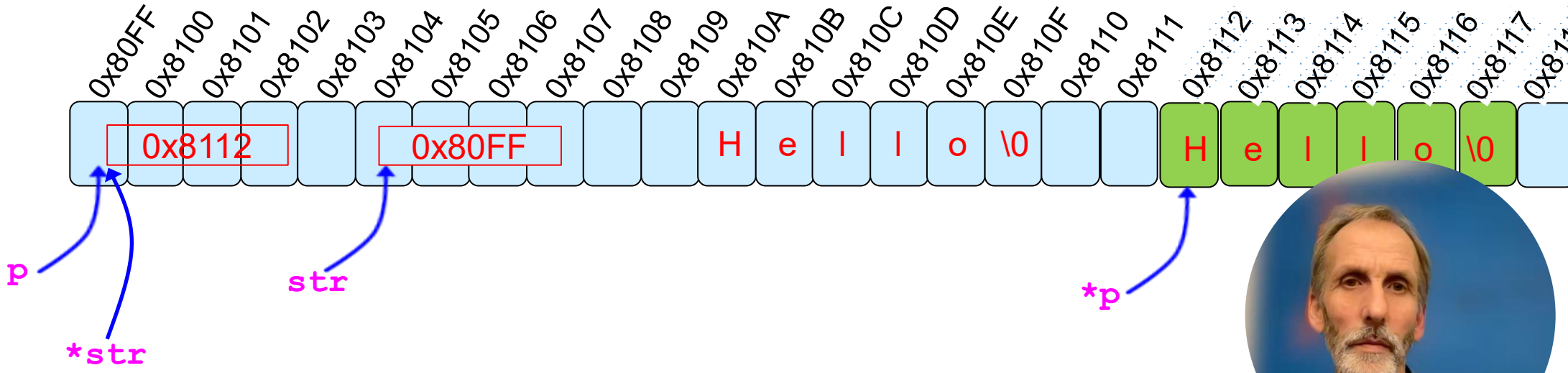
```
char  *p;
func(p);

void setstring(char *str)
*str = (char *)malloc(6);
strcpy(*str, "Hello");
strcpy(0x8112, 0x810A);
```





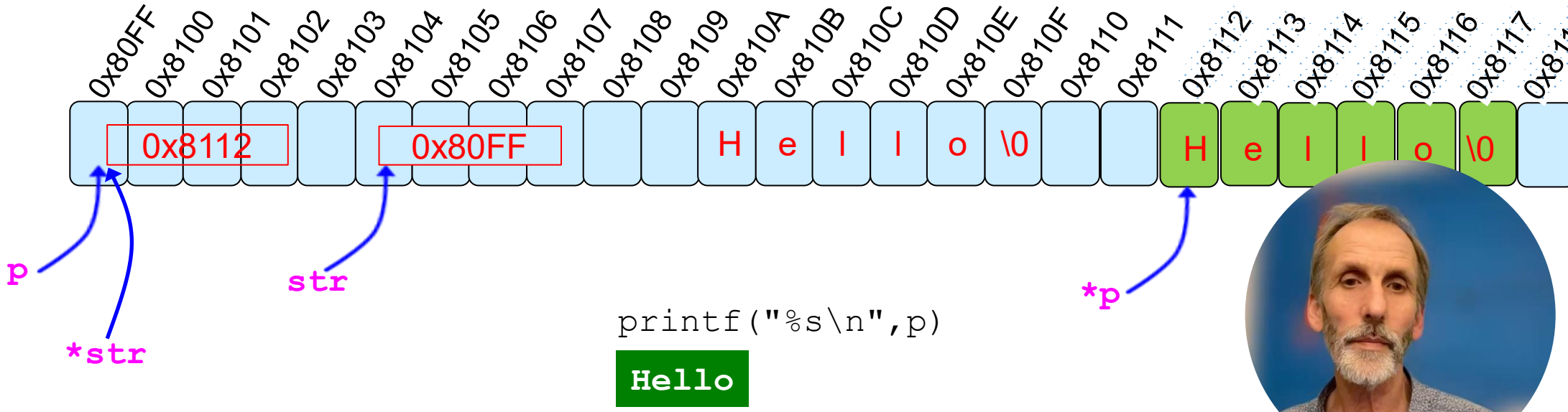
Functions: pass by reference

```
char  *p;
func(p);

void setstring(char *str)
*str = (char *)malloc(6);
strcpy(*str, "Hello");
strcpy(0x8112, 0x810A);
```



Functions: pass by reference

```
char  *p;
func(p);

void setstring(char *str)
*str = (char *)malloc(6);
strcpy(*str, "Hello");
strcpy(0x8112, 0x810A);
```




```
printf("%s\n",p)
```

Hello

Function parameters

University of Exeter

- Parameters to functions are always passed by value.
- To update the value of a parameter and pass the new value back to the calling function you must use pointers:
  - supply (as a pointer) the address of the variable you want to alter so the function can alter the variable by dereferencing the pointer.
- Since C functions can only return one value, this mechanism can be used to return other values to the caller.